# Department of Computer Science and Engineering
# Assignment Questions

## Course Code & Name : DSA01 - Object Oriented Programming with C++

| | |
|---|---|
| **Assignment Title:** | Urban Complaint Management and Service Request System |
| **Course Outcome(s) – CO:** | **CO3:** Construct C++ programs using constructors, destructors, and operator overloading mechanisms to control object creation, destruction, and operator behaviour .<br>**CO4:** Analyse and implement C++ applications using inheritance, types of inheritance, virtual base classes, abstract classes, pointers, and polymorphism to design reusable and extensible programs. |
| **Bloom's Taxonomy Level** | L3 – Apply; L4 – Analyse, L5-Evaluate |
| **SDG Mapping (SDG 1-17)** | SDG 9 – Industry, Innovation and Infrastructure;<br>SDG 11 – Sustainable Cities and Communities;<br>SDG 16 – Peace, Justice and Strong Institutions;<br>SDG 12 – Responsible Consumption and Production |

**Problem Statement:** Design, implement, and evaluate a menu-driven, object-oriented C++ application for an Urban Complaint Management and Service Request System to automate the registration, categorization, prioritization, assignment, status tracking, and resolution of civic complaints such as road damage, water supply, waste management, drainage, and streetlight issues. The system shall model citizens, complaints, departments, and service requests as encapsulated classes and manage multiple objects using suitable data types, control structures, arrays, and static members. Different complaint types shall be organized using inheritance and virtual functions to support runtime polymorphism and customized complaint processing. The application shall demonstrate default, parameterized, and copy constructors, destructors, pointers, and at least two meaningful operator

overloads for comparing complaints and generating reports. A menu-driven interface shall provide complaint registration, department assignment, status updates, complaint comparison, resolution tracking, and service-performance report generation.

**Assessment Rubrics**

| Criteria (CO Mapping) | Max Marks | Excellent | Good | Satisfactory | Needs Improvement |
|---|---|---|---|---|---|
| Requirement Understanding, Data Types, Operators & Control Structures (CO1) | 10 | Correctly identifies the complaint-management workflow; uses suitable data types, operators, conditional statements and loops efficiently. | Identifies most requirements with minor gaps; uses data types and control structures appropriately | Basic requirements identified; some errors or inefficiencies in data types and control structures | Requirements poorly understood; frequent errors in data types, operators or control flow. |
| Class Design: Encapsulation, Access Specifiers & Member Functions (CO2) | 15 | Well-designed classes for proper encapsulation, access specifiers and cohesive member functions. | Classes are appropriately designed with minor issues in encapsulation or member functions. | Classes are present but encapsulation is weak or member functions are partially implemented. | Poor class design; inappropriate access specifiers and inadequate separation of data and behavior. |
| Static Members & Arrays of Objects (CO2) | 10 | Correctly uses static members for shared complaint statistics and arrays/collections of objects to | Static members and arrays of objects are implemented with minor logical errors. | Static members or arrays are partially implemented or underused. | Static members and arrays of objects are missing or non-functional. |

| | | | | | |
|---|---|---|---|---|---|
| | | manage multiple citizens, complaints and requests. | | | |
| Inheritance Hierarchy & Polymorphism (Virtual Functions) (CO4) | 15 | Clearly implements an inheritance hierarchy for different complaint/service types; virtual functions effectively provide runtime polymorphism for complaint processing and priority handling. | Inheritance and virtual functions are correctly implemented with limited polymorphic behaviour. | Basic inheritance is implemented, but overriding or polymorphism is incomplete. | No meaningful inheritance or polymorphism is implemented. |
| Constructors, Destructors & Operator Overloading (CO3) | 15 | Correctly implements default, parameterized and copy constructors; destructor performs appropriate resource cleanup; at least two meaningful operator overloads are correctly implemented for complaint | Most constructors and at least one operator overload are correctly implemented with minor issues.. | Some constructor types or one operator overload is implemented; destructor functionality is incomplete. | Constructors, destructor and operator overloading are missing, incorrect or cause runtime errors.. |

| | | | | | |
|---|---|---|---|---|---|
| | | comparison/report formatting. | | | |
| Menu-Driven Functionality, Correctness & Report Generation | 25 | All features—complaint registration, categorization, prioritization, department assignment, status updates, resolution tracking, comparison and service reports—work correctly through a clear menu-driven interface. | Most features work correctly with minor logical or formatting errors. | Core functions work, but some features such as prioritization, tracking or reporting are incomplete. | Program is unreliable; major features are missing or non-functional. |
| Reflection: Design Decisions, SDG Relevance & Learning Outcomes | 10 | Thoughtful justification of design choices, clear connection to SDG 7/11/12 relevance, and honest, specific account of challenges faced and learning gained. | General justification of design choices; sustainability connection and challenges mentioned but lack depth. | Reflection present but generic; limited justification of design or vague account of learning outcomes. | No genuine reflection submitted, or content is generic with no real design/SDG/learning connection. |

## 1. Introduction

The Urban Complaint Management and Service Request System is a menu-driven, object-oriented C++ application designed to manage common civic complaints such as road damage, water supply problems and waste-management issues. The system allows citizens to be registered, complaints to be recorded, complaint priority to be calculated, departments to be assigned, statuses to be updated, complaints to be compared and service-performance information to be reported.

The implementation focuses on the object-oriented programming requirements of the assignment. It uses encapsulation, constructors, a copy constructor, a destructor, inheritance, virtual functions, runtime polymorphism, pointers, a static member, operator overloading and a collection of dynamically created complaint objects.

## 2. Objectives

- Register citizens and maintain their basic details.
- Register and categorize urban complaints.
- Calculate complaint priority using severity, impact, urgency and complaint type.
- Automatically recommend the appropriate service department.
- Update and track complaint status.
- Compare complaints based on priority using operator overloading.
- Generate a service-performance report.
- Demonstrate major C++ object-oriented programming concepts in a practical application.

## 3. System Features

| **Feature** | **Description** |
|---|---|
| Citizen Registration | Stores citizen ID and name. |

| | |
|---|---|
| Complaint Registration | Creates a complaint object for Road, Water or Waste issues. |
| Smart Priority | Computes a priority score from severity, affected people, urgency and complaint type. |
| Automatic Department | Maps complaint type to a recommended department. |
| Status Tracking | Supports Pending, In Progress and Resolved states. |
| Complaint Comparison | Compares priority scores using overloaded operators. |
| Service Report | Displays total, pending, in-progress and resolved complaints. |

4. Class Design

| **Class** | **Responsibility** |
|---|---|
| Citizen | Encapsulates citizen ID and name; demonstrates constructors and copying. |
| Complaint | Base class containing common complaint data, priority calculation, status and operators. |
| RoadComplaint | Derived class for road-related complaints. |
| WaterComplaint | Derived class for water-supply complaints. |

| | |
|---|---|
| WasteComplaint | Derived class for waste-management complaints. |

5. OOP Concepts Implemented

| **Concept** | **Implementation** |
|---|---|
| Encapsulation | Citizen data is private; complaint data is protected and accessed through member functions. |
| Default Constructor | Initializes objects with default values. |
| Parameterized Constructor | Creates complaint objects using supplied citizen, description and priority inputs. |
| Copy Constructor | Copies the data of an existing Citizen or Complaint object. |
| Destructor | Complaint has a virtual destructor; dynamically created complaint objects are deleted before program termination. |
| Inheritance | RoadComplaint, WaterComplaint and WasteComplaint inherit from Complaint. |
| Virtual Functions | Derived classes override type(), dept() and typeScore(). |
| Runtime Polymorphism | Complaint pointers refer to different derived complaint objects. |
| Pointers | Complaint* is used for dynamic object creation and lookup. |

| | |
|---|---|
| Static Member | Complaint::total stores the total number of registered complaints. |
| Operator Overloading | operator> compares priority and operator<< formats complaint output. |
| Vector | Vectors store multiple citizens and complaint pointers. |

6. Smart Priority Calculation

The system adds a simple decision-making feature instead of assigning a fixed priority. The score is calculated using severity, population impact, urgency and the complaint type score.

**Priority Score = Severity + Impact Score + Urgency + Type Score**

The impact score increases when more people are affected. Water complaints receive a higher type score in the current implementation because interruption of water supply is treated as more urgent. The resulting score is classified as LOW, MEDIUM, HIGH or CRITICAL.

7. Menu-Driven Workflow

1. Register Citizen
2. Register Complaint
3. View Complaints
4. Assign Department
5. Update Status
6. Compare Complaints
7. Service Report
8. Exit

8. Pseudocode

START

Create collections for citizens and complaints.

Display the main menu.

REPEAT

Read menu choice.

IF Register Citizen

Read citizen details.

Create Citizen object.

Store citizen.

ELSE IF Register Complaint

Find citizen.

Read complaint type and priority inputs.

Create the appropriate derived Complaint object.

Calculate priority.

Store complaint.

ELSE IF View Complaints

Display all complaint objects.

ELSE IF Assign Department

Find complaint.

Assign recommended department.

ELSE IF Update Status

Find complaint.

Change its status.

ELSE IF Compare Complaints

Find two complaints.

Compare their priority scores using operator>.

ELSE IF Service Report

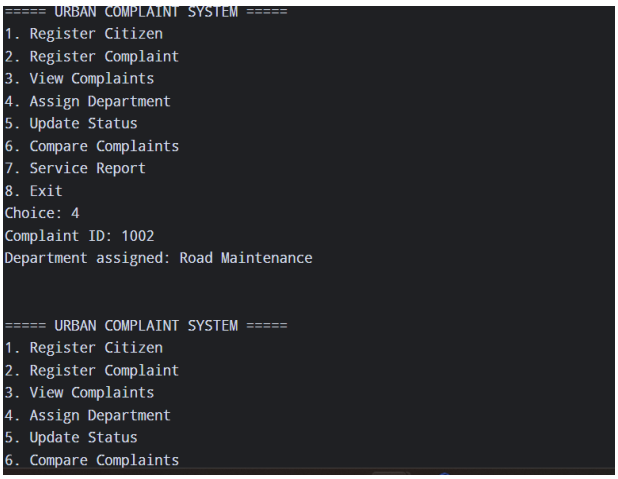
Count complaints by status.

Display statistics.

UNTIL Exit.

Delete dynamically allocated complaint objects.

END

```
Choice: 3

ID    Type        Status         Department           Score Level
-------------------------------------------------------
1001  Water       Pending        Not Assigned        16      CRITICAL
1002  Road        Pending        Not Assigned        11      HIGH


===== URBAN COMPLAINT SYSTEM =====
1. Register Citizen
2. Register Complaint
3. View Complaints
4. Assign Department
5. Update Status
6. Compare Complaints
7. Service Report
8. Exit
Choice: 4
Complaint ID: 1001
Department assigned: Water Supply
```

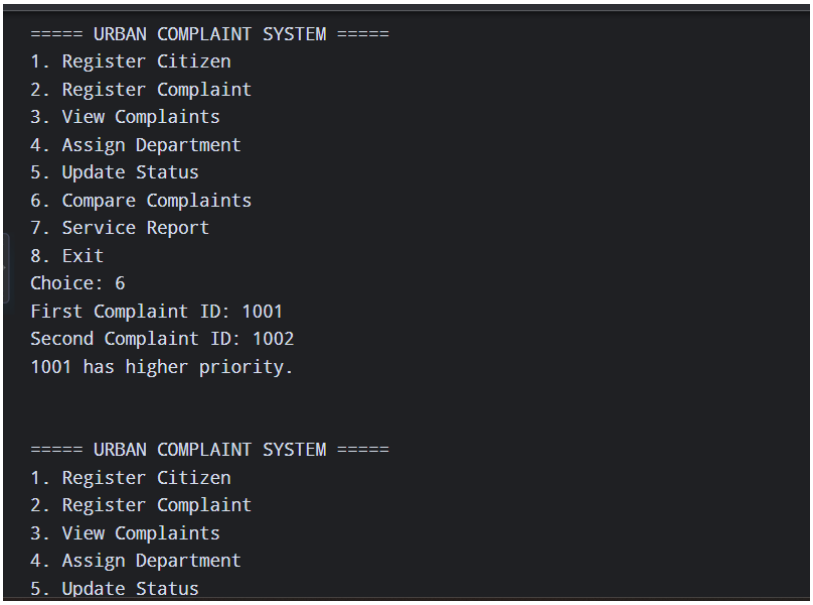
```
Choice: 2
Citizen ID: 102

1.Road  2.Water  3.Waste
Type: 1
Description: roadDamage
Severity (1-5): 3
Affected people: 20
Urgency (1-5): 3

Complaint Registered!
Priority Score: 11
Priority Level: HIGH


===== URBAN COMPLAINT SYSTEM =====
1. Register Citizen
2. Register Complaint
3. View Complaints
4. Assign Department
5. Update Status
```

```
1.Road  2.Water  3.Waste
Type: 2
Description: Watersupply
Severity (1-5): 5
Affected people: 100
Urgency (1-5): 5

Complaint Registered!
Priority Score: 16
Priority Level: CRITICAL


===== URBAN COMPLAINT SYSTEM =====
1. Register Citizen
2. Register Complaint
3. View Complaints
4. Assign Department
5. Update Status
6. Compare Complaints
7. Service Report
8. Exit
```

```
7. Service Report
8. Exit
Choice: 1
Citizen Name: Priya
Citizen registered.


===== URBAN COMPLAINT SYSTEM =====
1. Register Citizen
2. Register Complaint
3. View Complaints
4. Assign Department
5. Update Status
6. Compare Complaints
7. Service Report
8. Exit
Choice: 2
Citizen ID: 101
```

```
Output

===== URBAN COMPLAINT SYSTEM =====
1. Register Citizen
2. Register Complaint
3. View Complaints
4. Assign Department
5. Update Status
6. Compare Complaints
7. Service Report
8. Exit
Choice: 1
Citizen Name: Ravi
Citizen registered.


===== URBAN COMPLAINT SYSTEM =====
1. Register Citizen
2. Register Complaint
3. View Complaints
4. Assign Department
5. Update Status
6. Compare Complaints
```

```
7. Service Report
8. Exit
Choice: 7

===== SERVICE REPORT =====
Total Complaints : 2
Pending          : 0
In Progress      : 1
Resolved         : 1


===== URBAN COMPLAINT SYSTEM =====
1. Register Citizen
2. Register Complaint
3. View Complaints
4. Assign Department
5. Update Status
6. Compare Complaints
7. Service Report
8. Exit
```




## 9. Reflection and Learning Outcomes

This project helped demonstrate how C++ OOP concepts can be combined to solve a real-world civic-service problem. The use of a base Complaint class and derived complaint types reduces repetition and allows runtime polymorphism. Constructors make object creation organized, while the destructor and delete operations demonstrate memory cleanup.

The smart priority calculation was selected as a practical enhancement because it makes complaint handling more meaningful than simply storing records. A major learning point was understanding how a base-class pointer can refer to different derived objects and invoke overridden functions at runtime.

## 10. SDG Relevance

The assignment maps the project to SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities; SDG 16 – Peace, Justice and Strong Institutions; and SDG 12 – Responsible Consumption and Production.

The strongest connection is SDG 11 because the application provides a structured way to register, prioritize and track urban service complaints, supporting better management of community issues.

## 11. Conclusion

The Urban Complaint Management and Service Request System provides a compact but feature-rich C++ implementation for civic complaint handling. It combines a menu-driven interface with inheritance, runtime polymorphism, constructors, destructor, pointers, static members, operator overloading and smart priority calculation. The system can be extended in the future with additional complaint types, persistent file storage, authentication or a graphical/web interface.